ระบบฐานข้อมูลคนพิการ มจธ.
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

เอกสารส่งมอบระบบ · System Documentation

เอกสารวิเคราะห์และออกแบบระบบ (SDD)

ระบบฐานข้อมูลคนพิการ โครงการฝึกอบรม-ฝึกงานคนพิการ (PWDs Database)

สถาปัตยกรรม เทคโนโลยีและเหตุผล API ตรรกะสำคัญ ความปลอดภัยและความเป็นส่วนตัว และการสำรองข้อมูล

SA | ระบบเวอร์ชัน v1.11.0 | kmutt-pwd.duckdns.org

เวอร์ชันเอกสาร	วันที่จัดทำ	ระยะเวลาโครงการ	สถานะระบบ
1.0	27 ตุลาคม 2569	23 พ.ค. – 27 ต.ค. 2569	ใช้งานจริงแล้ว

การควบคุมเอกสาร

เวอร์ชัน	วันที่	รายละเอียด	ผู้จัดทำ
1.0	27 ตุลาคม 2569	จัดทำครั้งแรก — ตรงกับระบบเวอร์ชัน v1.11.0 (commit b2df9ed)	ทีมพัฒนาระบบ

ขอบเขตของเอกสารฉบับนี้

สถาปัตยกรรม เทคโนโลยีและเหตุผล API ตรรกะสำคัญ ความปลอดภัยและความเป็นส่วนตัว และการสำรองข้อมูล — เป็นส่วนหนึ่งของชุดเอกสารโครงการ (ฉบับรวมมีครบทุกบท และใช้เลขบทเดียวกัน)

เอกสารในชุดนี้

ไฟล์	สำหรับ	เนื้อหา
เอกสารประกอบการพัฒนาระบบ (ฉบับรวม)	BA · SA · Design · Dev · Test · Ops	ทุกระยะของโครงการ ตั้งแต่เก็บความต้องการ วิเคราะห์ ออกแบบ พัฒนา ทดสอบ จนถึงติดตั้งและดูแลระบบจริง
เอกสารความต้องการของระบบ (BRD / SRS)	BA	ที่มา ขอบเขต เอกสารความต้องการจากหน่วยงาน ผู้ใช้และสิทธิ์ กรณีการใช้งาน ความต้องการเชิงหน้าที่ กฎทางธุรกิจ และบันทึกการเปลี่ยนแปลงความต้องการ
เอกสารวิเคราะห์และออกแบบระบบ (SDD)	SA	สถาปัตยกรรม เทคโนโลยีและเหตุผล API ตรรกะสำคัญ ความปลอดภัยและความเป็นส่วนตัว และการสำรองข้อมูล
เอกสารออกแบบฐานข้อมูลและ UX/UI (Database & UI Design)	Design	แผนภาพฐานข้อมูล พจนานุกรมข้อมูล หลักการ UX ระบบการออกแบบ แผนผังหน้าเว็บ ลำดับการใช้งาน และหน้าจอจริงทุกหน้า
เอกสารแผนและผลการทดสอบ (Test Plan & Report)	Test	กลยุทธ์การทดสอบ สภาพแวดล้อม ผลการทดสอบจริง กรณีทดสอบทั้งหมด ข้อบกพร่องที่พบ และแบบฟอร์มทดสอบการยอมรับ (UAT)
เอกสารการพัฒนาและติดตั้งระบบ (Development & Operations)	Dev · Ops	โครงสร้างโปรแกรม มาตรฐานการพัฒนา เวอร์ชัน การติดตั้ง สำรองและกู้ข้อมูล การเฝ้าระบบ คู่มือผู้ดูแลระบบ และงานที่ค้าง

ที่มาของข้อมูลในเอกสาร ตาราง/คอลัมน์ดึงจากโครงสร้างฐานข้อมูลจริง (schema.prisma) · ผลการทดสอบดึงจากผลรันจริงล่าสุด · ภาพหน้าจอถ่ายจากระบบจริงด้วย**ข้อมูลสมมติ** (ไม่มีข้อมูลคนพิการจริงในเอกสาร) · ความต้องการมาจากเอกสารที่หน่วยงานส่งมา 6 ฉบับ และบันทึกการพัฒนา (CHANGELOG)

การลงนามรับรองเอกสาร

ผู้จัดทำ	ผู้ตรวจสอบ	ผู้อนุมัติ (เจ้าของระบบ)
วันที่ ...../...../.....	วันที่ ...../...../.....	วันที่ ...../...../.....

สารบัญ

บทที่ 3 การวิเคราะห์และออกแบบระบบ SA

3.1 สถาปัตยกรรมระบบ
3.2 เทคโนโลยีที่เลือกและเหตุผล
3.3 โครงสร้างโมดูล
3.4 การออกแบบ API
3.5 ตรรกะสำคัญ
3.6 ความปลอดภัยและความเป็นส่วนตัว
3.7 การแจ้งเตือน
3.8 การสำรองและกู้คืนข้อมูล

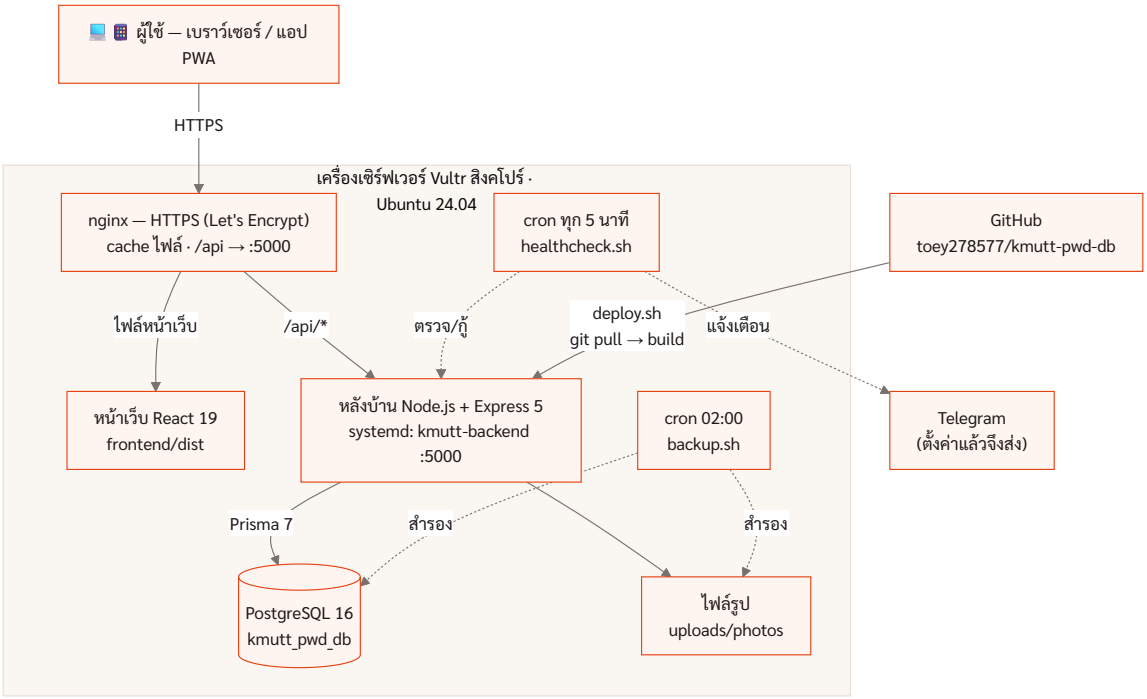
SA

3 การวิเคราะห์และออกแบบระบบ

บทนี้อธิบายว่าระบบสร้างอย่างไร — โครงสร้าง เทคโนโลยีที่เลือกและเหตุผล รายการ API ตรรกะสำคัญ ความปลอดภัย และการสำรองข้อมูล ทุกอย่างอ้างจากโค้ดจริงของระบบเวอร์ชันปัจจุบัน

3.1 สถาปัตยกรรมระบบ

เป็นเว็บแอปแบบ **3 ชั้น** (หน้าเว็บ → หลังบ้าน API → ฐานข้อมูล) ทั้งหมดอยู่บนเครื่องเซิร์ฟเวอร์เครื่องเดียวของหน่วยงาน มี nginx เป็นประตูหน้า (HTTPS) เสิร์ฟหน้าเว็บที่ build แล้ว และส่งต่อคำขอ `/api/` ไปที่หลังบ้าน



ภาพที่ 3.1 สถาปัตยกรรมระบบ (เครื่องเดียวกันมีระบบจองห้องประชุมอยู่ด้วย แยกโฟลเดอร์ พอร์ต และฐานข้อมูล)

3.2 เทคโนโลยีที่เลือกและเหตุผล

ส่วน	เทคโนโลยี	เหตุผล
หน้าเว็บ	React 19 · Vite 8 · React Router 7	หน้าเว็บแบบแอป ลื่น เปลี่ยนหน้าไม่ต้องโหลดใหม่ · โหลดทีละหน้า (lazy) · ชุมชนใหญ่ หาคนดูแลต่อง่าย
หน้าตา	Tailwind CSS 4 · DaisyUI 5 · lucide-react	ทำธีมสีเดียวทั้งระบบจากที่เดียว (ชุดสีส้มโลโก้ใน `@theme`) · รองรับมือถือ · ไอคอนครบ
ไฟล์ Excel	SheetJS (xlsx)	อ่าน/สร้างไฟล์ .xlsx ในเบราว์เซอร์ — นำเข้า/ส่งออกไม่ต้องส่งไฟล์ขึ้นเซิร์ฟเวอร์ก่อนตรวจ
แอปบนมือถือ	vite-plugin-pwa	ติดตั้งเป็นไอคอนบนมือถือ/คอมได้ · แจ้ง "มีเวอร์ชันใหม่" เมื่ออัปเดต
หลังบ้าน	Node.js + Express 5	ภาษาเดียวกับหน้าเว็บ (JavaScript) · เบา เหมาะกับเครื่อง RAM 1 GB

ส่วน	เทคโนโลยี	เหตุผล
ฐานข้อมูล	PostgreSQL 16 + Prisma 7	ฐานข้อมูลเชิงสัมพันธ์ตามโครงสร้าง 10+ ตารางของหน่วยงาน · Prisma คุมโครงสร้างด้วย migration ย้อนดูได้ทุกครั้ง
ความปลอดภัย	bcryptjs · jsonwebtoken · helmet · express-rate-limit	เข้ารหัสรหัสผ่าน · ตั๋วเข้าระบบ · security headers · จำกัดการเดารหัส
เครื่อง	VPS Vultr $5/เดือน · nginx · Let's Encrypt · systemd · cron	ข้อมูลอยู่ในมือหน่วยงาน (หลังเสียข้อมูลจากบริการฟรี 2 ครั้ง) · ต้นทุนต่ำ · ตั้งสำรองข้อมูลเองได้

บทเรียนสำคัญ ช่วง พ.ค.–ก.ค. 2569 ระบบอยู่บนบริการฟรี (Vercel + Render + Neon) — ฐานข้อมูล Render หมดอายุ และต่อมาตัวปลุกเซิร์ฟเวอร์ที่ยิง `/api/health` (แตะฐานข้อมูล) ทุก 5 นาทีทำให้โควต้า Neon หมด ดึงข้อมูลออกไม่ได้ จึงย้ายขึ้น VPS (20 ก.ค.) เพิ่ม `/api/ping` ที่ไม่แตะฐานข้อมูล และตั้งสำรองข้อมูลทุกคืน

3.3 โครงสร้างโมดูล

โมดูล	หลังบ้าน (backend/src)	หน้าเว็บ (frontend/src)
เข้าสู่ระบบ / สิทธิ์	routes/auth.js · middleware/auth.js (authenticate, requireRole) · utils/jwtSecret.js	context/AuthContext.jsx · pages/Login.jsx · components/SessionGuard.jsx · utils/session.js
คนพิการ + รายการย่อย	routes/persons.js · training · workexp · followup · skills · personorg · photos · assessments · utils/owned.js · utils/uploadDir.js	pages/PersonList.jsx · PersonDetail.jsx · utils/name.js · useThaiDate.js · resizeImage.js
นำเข้า Excel	POST /persons/import · utils/province.js · utils/text.js	utils/importCheck.js · utils/province.js (ตรวจก่อนส่ง)
รุ่น / ประเมิน	routes/batches.js · assessments.js	pages/BatchPage.jsx · TrainingList.jsx · แท็บประเมินใน PersonDetail
ติดตามผล	routes/aggregate.js (followup-all, followup-due) · utils/followupDue.js	pages/FollowUpList.jsx · components/FollowUpDue.jsx
ข้อมูลยังไม่ครบ	GET /data/incomplete · utils/incomplete.js	pages/DataCheck.jsx · ตัวเลขใน components/Layout.jsx · การ์ดใน Dashboard
Dashboard / รายงาน	routes/dashboard.js	pages/Dashboard.jsx (กราฟ CSS) · ReportPage.jsx (พิมพ์/PDF/Excel)
ผู้ดูแลระบบ	routes/users.js · audit.js · disabilityTypes.js · system.js · middleware/audit.js	pages/UserManagement.jsx · AuditLog.jsx · DisabilityTypes.jsx · components/DangerZone.jsx
ส่วนกลาง	index.js (helmet, rate limit, แปลง error เป็นภาษาไทย, ล้าง cache) · prismaClient.js	App.jsx (router lazy) · api/index.js (axios + cache) · Layout · ToastContext · ThemeContext · UpdateBanner · ErrorBoundary · LoadError · MobileCard · DiscardBar

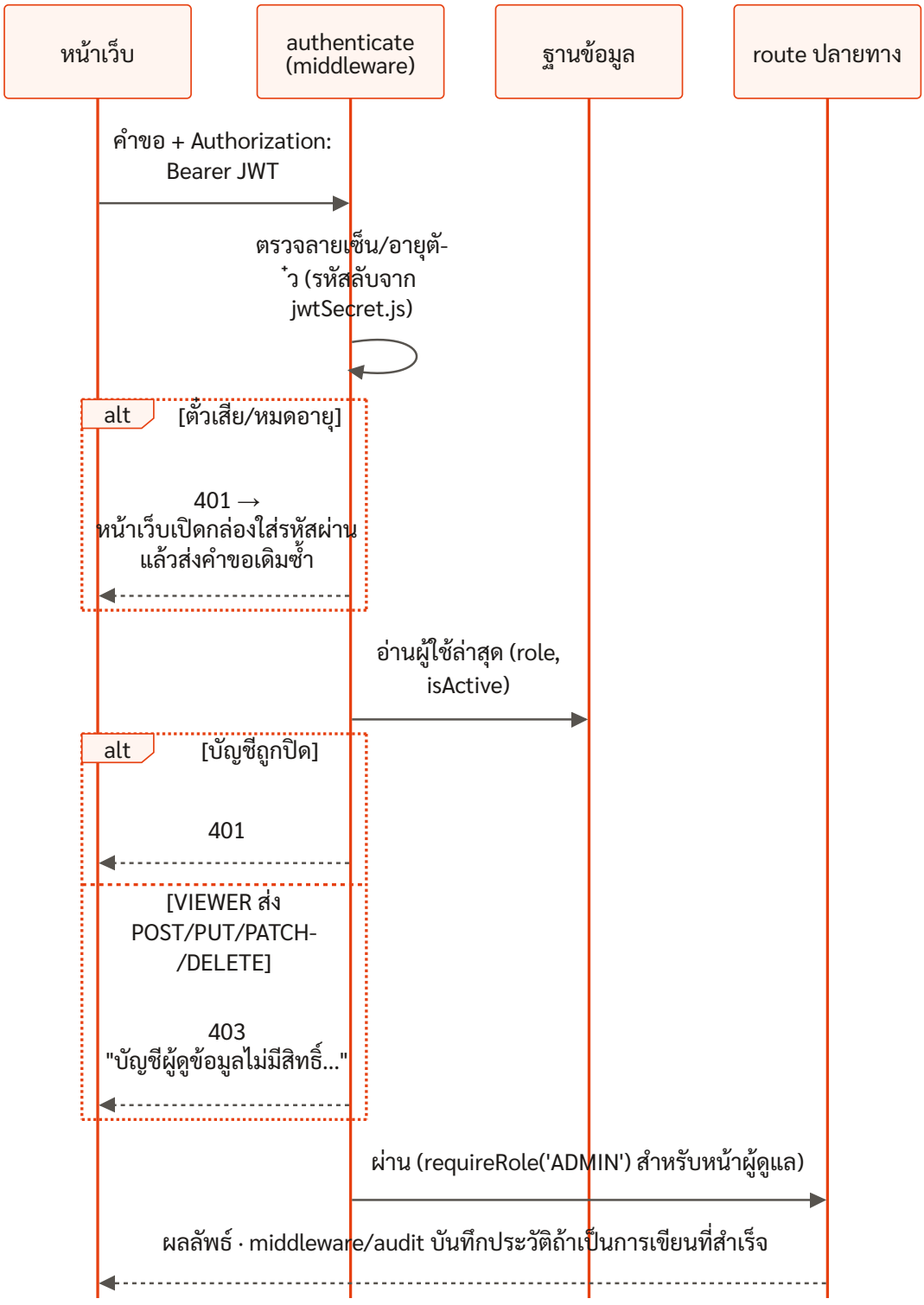
3.4 การออกแบบ API

REST API ภายใต้ `/api` รับส่ง JSON · ทุกเส้นทางต้องแนบตั๋ว `Authorization: Bearer <JWT>` ยกเว้นที่ระบุ · ตอบข้อผิดพลาดเป็น `{ error: "ข้อความภาษาไทย" }` พร้อมรหัส 400 / 401 / 403 / 404 / 429

กลุ่ม	เส้นทาง	ทำอะไร	สิทธิ์
เข้าสู่ระบบ	`POST /api/auth/login`	เข้าสู่ระบบ → ตั๋ว JWT 8 ชม. (ล็อกอินผิด 8 ครั้ง/15 นาที/IP = บล็อก)	ไม่ต้องล็อกอิน
	`GET /api/auth/me` · `GET /api/auth/refresh`	ข้อมูลตัวเองล่าสุดจากฐาน · ต่อเวลาอีก 8 ชม. (ต้องมีตั๋วที่ยังไม่หมด)	ทุกบทบาท
คนพิการ	`GET /api/persons`	รายชื่อ + ค้นหา (?search=) + ตัวกรอง batchId · gender · province · disabilityTypeId · employment (EMPLOYED/UNEMPLOYED/STUDYING/NONE) · withSkills · ส่งผลติดตามล่าสุดมาด้วย	ทุกบทบาท
	`POST /api/persons` · `GET/PUT/DELETE /api/persons/:id`	เพิ่ม · ดูทั้งหมด · แก้ · ลบ (ลบไฟล์รูปตามด้วย)	เขียน: เจ้าหน้าที่/แอดมิน
	`POST /api/persons/import`	นำเข้าหลายคน — ตรวจรุ่น/หลักสูตร/ประเภทความพิการ/จังหวัดซ้ำที่หลังบ้าน ผิด = 400 ไม่นำเข้าเลย	เจ้าหน้าที่/แอดมิน
รายการย่อยของคน	`/api/persons/:id/{disability · training · workexp · followup · skills · personorg · photos · assessments}[/:subId]`	GET รายการ · POST เพิ่ม · PUT แก้ (ยกเว้นรูป/ผลประเมิน) · DELETE ลบ — ต้องเป็นของคนใน URL จริง (ensureOwned) รับเฉพาะช่องที่อนุญาต (pick)	เขียน: เจ้าหน้าที่/แอดมิน
	`GET /api/persons/disability-types` · `GET /api/persons/batch/:batchId/assessments`	ตัวเลือกประเภทความพิการ · ผลประเมินทั้งรุ่น	ทุกบทบาท
ข้อมูลรวม	`GET /api/data/{training-all · batch-enrollments · followup-all · followup-due · incomplete}`	ประวัติอบรมทุกคน · คนในรุ่น + คะแนน · ติดตามผลทุกคน · ใครถึงรอบติดตามผล · ใครข้อมูลยังไม่ครบ	ทุกบทบาท
Dashboard	`GET /api/dashboard/stats`	สถิติทั้งหมดของหน้าแรก (จำผล 60 วิ ล้างทันทีเมื่อมีการเขียนข้อมูล)	ทุกบทบาท
สถานประกอบการ	`GET/POST /api/organizations` · `GET/PUT/DELETE /api/organizations/:id`	จัดการองค์กร (ลบแล้วลบการเชื่อมกับคนด้วย)	เขียน: เจ้าหน้าที่/แอดมิน
รุ่นและหลักสูตร	`GET/POST /api/batches` · `GET/PUT/DELETE /api/batches/:id` · `POST/PUT/DELETE /api/batches/:id/courses[/:courseId]`	จัดการรุ่น + หลักสูตร (สูงสุด 4 ต่อรุ่น)	เขียน: เจ้าหน้าที่/แอดมิน
ประเภทความพิการ	`GET/POST /api/disability-types` · `PUT/DELETE /api/disability-types/:id`	อ่าน (+ จำนวนคนที่ใช้) · เพิ่ม/แก้/ลบ (ลบได้เฉพาะที่ไม่มีคนใช้)	อ่าน: ทุกบทบาท · เขียน: แอดมิน
ผู้ดูแลระบบ	`GET/POST /api/users` · `PUT/DELETE /api/users/:id`	จัดการผู้ใช้ (กันแอดมินคนสุดท้าย / แก้สิทธิ์ตัวเอง)	แอดมิน
	`GET /api/audit-logs`	ประวัติการใช้งาน ค้นหา/กรอง (?personId= สำหรับแท็บในหน้าคน)	แอดมิน
	`GET /api/system/stats` · `POST /api/system/reset`	จำนวนข้อมูลที่จะลบ · รีเซ็ตระบบ (คำยืนยัน + รหัสผ่าน)	แอดมิน
ระบบ	`GET /api/health` · `GET /api/ping` · `GET /api/uploads/…`	ตรวจระบบ (แตะฐานข้อมูล) · ตรวจว่าเว็บตอบ (ไม่แตะฐาน) · ไฟล์รูป	ไม่ต้องล็อกอิน

3.5 ตรรกะสำคัญ

3.5.1 การตรวจสิทธิ์ทุกคำขอ

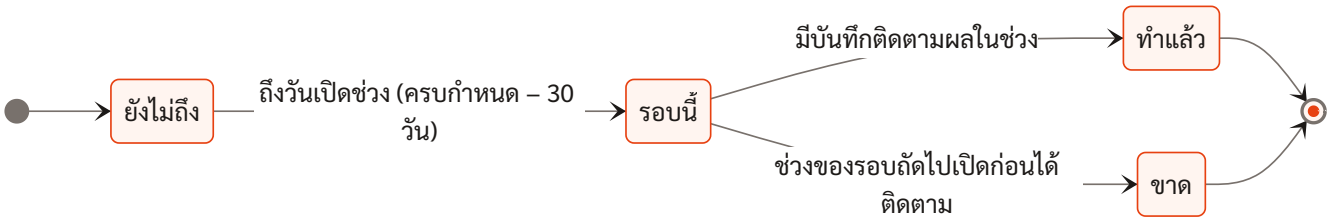


ภาพที่ 3.2 ลำดับการตรวจสิทธิ์ — อ่านสิทธิ์จากฐานทุกครั้ง จึงเปลี่ยนสิทธิ์/ปิดบัญชีมีผลทันที

3.5.2 รอบติดตามผล (utils/followupDue.js)

- ครบกำหนดแต่ละรอบ = **วันสิ้นสุดของรุ่น + 3 / 6 / 12 เดือน** (บวกเดือนแบบไม่ล้น: 31 ม.ค. + 1 เดือน = 28/29 ก.พ.)

- แต่ละรอบ "เปิด" ก่อนครบกำหนด 30 วัน และปิดเมื่อรอบถัดไปเปิด — บันทึกติดตามผลที่ลงวันที่ในช่วงไหน = ทำรอบนั้นแล้ว (ติดตามเดือนที่ 7 นับเป็นรอบ 6 เดือน รอบ 3 เดือนถือว่าขาด)
- รอบปัจจุบันยังไม่ทำ → สถานะ **ใกล้ถึง** (อีก X วัน) หรือ **เลยกำหนด** (X วัน) เรียงเลยกำหนดนานสุดก่อน · รอบ 1 ปีเลยมาเกิน 180 วันเลิกเตือน
- ไม่นับคนเสียชีวิต · รุ่นที่ยังอบรมอยู่ไม่เตือน · รุ่นที่เสร็จสิ้นแล้วแต่ไม่มีวันสิ้นสุด → เตือนให้ใส่วันสิ้นสุด · "วันนี้" คิดตามเวลาไทย



ภาพที่ 3.3 สถานะของแต่ละรอบติดตามผล (ชิปในหน้าจอ: ทำแล้ว / ขาด / รอบนี้ / ยังไม่ถึง)

3.5.3 นำเข้า Excel แบบตรวจเข้ม

- **ชั้นที่ 1 — หน้าเว็บ** (utils/importCheck.js, province.js): อ่านไฟล์ในเบราว์เซอร์ ตรวจทุกแถวทันทีที่เลือกไฟล์ → กล่องแดง (ต้องแก้: แถว ชื่อ ช่อง สิ่งที่พิมพ์ เหตุผล+วิธีแก้) / กล่องเหลือง (แก้ให้อัตโนมัติ หรือเตือนว่าไม่ได้ใส่) · มีแถวแดง = ปุ่มนำเข้ากดไม่ได้
- แก้ให้อัตโนมัติเฉพาะที่ไม่กำกวม: จังหวัดชื่อย่อ (กทม. → กรุงเทพมหานคร) · หลักสูตรต่างแค่ช่องว่าง/ตัวพิมพ์ · ประเภทความพิการพิมพ์ย่อที่ตรงได้ประเภทเดียว · วันเกิดปี พ.ศ. → ค.ศ.
- **ชั้นที่ 2 — หลังบ้าน**: ตรวจรุ่น หลักสูตร (ต้องอยู่ในรุ่นนั้น) ประเภทความพิการ และจังหวัดซ้ำ ผิด = 400 ไม่นำเข้าสักแถว · สร้างคน + ความพิการในคำสั่งเดียว (ไม่ค้างครึ่งๆ) · ตัดความยาวตามขนาดคอลัมน์ · ล้างตัวอักษรล่องหน

3.5.4 ข้อมูลยังไม่ครบ · สถิติ Dashboard

- ตัวตรวจ 7 ช่องอยู่ที่เดียว (utils/incomplete.js) ใช้ทั้งหน้า "ข้อมูลยังไม่ครบ" ตัวเลขข้างเมนู และการ์ด Dashboard — ตรงกับช่องที่ฟอร์มแก้ไขบังคับ
- สถานะการมีงานทำ/งานล่าสุด = ผลติดตามล่าสุดของแต่ละคน นับเป็นคน · ผู้เข้าอบรม = อยู่ในรุ่นหรือมีประวัติอบรม · ภูมิภาค 6 กลุ่มจาก 77 จังหวัด (จับชื่อย่อ/สะกดผิดที่พบบ่อย) · ช่วงอายุ ต่ำกว่า 15 / 15–25 / 26–35 / 36–45 / 46+ · วุฒิการศึกษารวมกลุ่มแบบไม่สนช่องว่าง/ตัวล่องหน
- หลังบ้านจำผลสถิติ 60 วิ หน้าเว็บจำผลข้อมูลไม่ครบ 30 วิ — ทั้งสองล้างทันทีเมื่อมีการเพิ่ม/แก้/ลบสำเร็จ ตัวเลขจึงตรงทันที

3.5.5 เวลาเข้าระบบ 8 ชั่วโมง

- เหลือไม่ถึง 10 นาที → แถบเตือน "ใช้งานต่อ" (ต่อเวลาด้วย /auth/refresh)
- หมดระหว่างใช้งาน → คำขอได้ 401 → กล่องใส่รหัสผ่านขึ้นทับหน้าเดิม (หลายคำขอพร้อมกัน = กล่องเดียว) → ล็อกอินแล้วส่งคำขอที่ค้างต่อให้เอง ข้อมูลในฟอร์มไม่หาย
- เปิดหน้าใหม่ตอนตั๋วหมดแล้ว → ไปหน้าเข้าสู่ระบบ · ตั๋วเก็บใน sessionStorage (ปิดแท็บ = ออกจากระบบ)

3.6 ความปลอดภัยและความเป็นส่วนตัว

ด้าน	สิ่งที่ทำ
การเข้าถึง	HTTPS ทุกหน้า (Let's Encrypt) · ตั๋ว JWT 8 ชม. · รหัสลับตั๋วจาก `JWT_SECRET` หรือสุ่มเก็บในไฟล์ `backend/.jwt-secret` (ไม่ขึ้น git) — ค่าเก่าที่เคยเขียนในโค้ดใช้ไม่ได้แล้ว · สิทธิ์อ่านจากฐานทุกคำขอ
รหัสผ่าน	เข้ารหัส bcrypt (ดูย้อนกลับไม่ได้) · อย่างน้อย 8 ตัว · ช่องตั้งรหัสในหน้าจัดการผู้ใช้พิมพ์ได้เฉพาะตัวอักษรอังกฤษ (กันลืมสลับแป้น) · ลืมรหัส = แอดมินตั้งใหม่ หรือ `npm run reset-admin` บนเครื่อง
กันการโจมตี	จำกัดคำขอ 2,000 ครั้ง/15 นาที/IP · ล็อกอินผิด 8 ครั้ง/15 นาที · helmet (กัน clickjacking/MIME sniffing ซ่อน X-Powered-By) · `trust proxy` ชั้นเดียว (เห็น IP จริง ปลอม X-Forwarded-For ไม่ได้) · รับข้อมูลไม่เกิน 8 MB · ไฟร์วอลล์ ufw เปิด 22/80/443 · SSH ใช้กุญแจเท่านั้น
ความถูกต้องของข้อมูล	รายการย่อยต้องเป็นของคนใน URL (ensureOwned) และรับเฉพาะช่องที่อนุญาต (pick — ไม่ส่ง req.body ตรงเข้าฐาน) · ข้อผิดพลาดจากฐานข้อมูลแปลงเป็นข้อความไทยที่จุดเดียว ของจริงเก็บใน log
ข้อมูลส่วนบุคคล (PDPA)	ประวัติการใช้งานทุกการเพิ่ม/แก้/ลบ + ล็อกอิน (ไม่เก็บเนื้อหาคำขอ) · รูปเก็บเป็นไฟล์ชื่อสุ่ม 32 ตัว (เดา URL ไม่ได้) · รูป/ไฟล์สำรองไม่ขึ้น git · เอกสารและชุดทดสอบใช้ข้อมูลสมมติ (เลขบัตร 88888888xxxxx)
การลบ	ลบคน = ลบรายการย่อยทั้งหมด (cascade) + ไฟล์รูป · รีเซ็ตระบบ 4 ด่าน · ไม่ลบประวัติการใช้งาน

3.7 การแจ้งเตือน

- **ในระบบ:** แจ้งรอบติดตามผล (Dashboard + หน้าติดตามผล) · ตัวเลขข้อมูลยังไม่ครบข้างเมนู · แถบ "มีเวอร์ชันใหม่ของระบบ" · แถบเตือนเวลาเข้าระบบใกล้หมด · Toast ผลการบันทึกทุกครั้ง
- **ของผู้ดูแลเครื่อง:** healthcheck.sh ส่งเข้า Telegram เมื่อเว็บล่มแล้วกู้ไม่ได้/กู้แล้ว ดิสก์ใกล้เต็ม สำรองข้อมูลหยุด — แจ้งครั้งเดียวต่อปัญหา · **ยังไม่ได้ตั้งค่า token** (ใส่ที่ `/opt/kmutt/.alert` เมื่อไรก็ส่งทันที)

3.8 การสำรองและกู้คืนข้อมูล

เรื่อง	รายละเอียด
สำรองอัตโนมัติ	cron ทุกวัน 02:00 → `backup.sh` : ฐานข้อมูลทุกตารางเป็นไฟล์ JSON (`scripts/backup-db.cjs`) + โฟลเดอร์รูปเป็น tar.gz · เก็บ 14 วัน ลบที่เก่ากว่าเอง · log `/var/log/kmutt-backup.log`
ก่อนรีเซ็ตระบบ	เลือกสำรองได้ → `backups/before-reset-<เวลา>.json` (เก็บ 5 ไฟล์ล่าสุด)
กู้คืน	`scripts/restore-db.cjs <ไฟล์>` — นับแถวที่กู้ได้จริง บอกต่อตารางว่าข้ามกี่แถว เตือนถ้ากู้ไม่ครบ · ขั้นตอนเต็มใน `docs/deploy/กู้ข้อมูล.md` (8 ขั้น มีขั้นสำรองสภาพปัจจุบันก่อน)
ซ้อมกู้	ชุดทดสอบ `api/backup-restore` สำรอง → ทำข้อมูลพัง → กู้ → ตรวจว่า 15 ตาราง ทุกแถว ทุกช่อง และรูปกลับมาเหมือนเดิม — รันทุกครั้งที่รันชุดทดสอบ
ข้อจำกัด	ไฟล์สำรองอยู่บนเครื่องเดียวกับระบบ ยังไม่มีสำเนานอกเครื่อง (ดูหัวข้อ 7.7)